

Learning to Prune

Weight-conditioned hypernetwork pruners

Major results

Hypernetworks read out a network's redundancy near-deterministically, beat hand-designed heuristics, and reveal redundancy as a structural property of width — not size, not depth.

Project diary · experiments/latest · 2026-06

The setup

What's the pruner? What are we measuring?

Method

- ▶ BiLSTM 'hypernetwork' reads the weights of one hidden layer at a time (row-by-row) and emits a 0/1 keep/kill mask per neuron.
- ▶ Trained jointly with CE on the frozen base model + a soft sparsity penalty $\lambda \cdot \text{mean per-layer gate}$. Pruner has its own parameters; base weights are untouched.
- ▶ Input = weight matrix only. No activations, no gradients, no data at inference. At deploy time: feed the frozen weights once, take the mask.

Metric

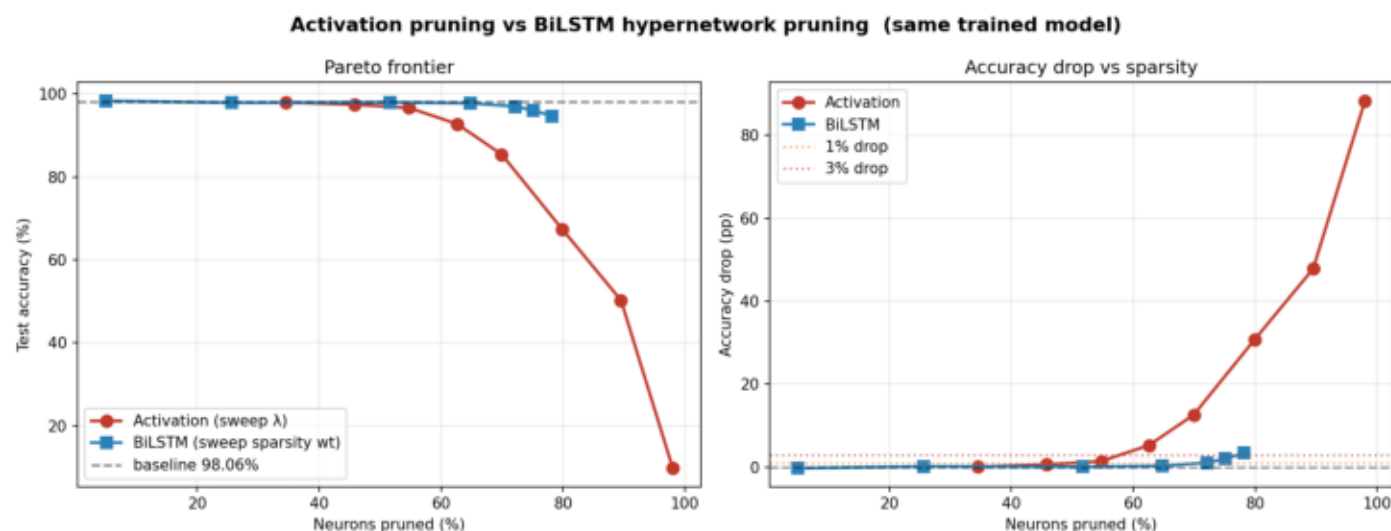
- ▶ % neurons pruned at iso-accuracy budget (we report $\leq 2\text{pp}$ drop or the Pareto curve directly).
- ▶ Pruning efficiency = $(\% \text{ pruned}) / \max(\text{drop_pp}, 0.5)$. One scalar to pick λ .

Bases studied

- ▶ MNIST MLPs — 4 shapes at ~2048 hidden neurons (narrow, deep, medium, wide). CIFAR-10 — LeNet (63K params) and a 'big' conv-net (10.4M).

BiLSTM strictly dominates heuristic pruning

Same MNIST [1024,1024] base, Pareto-traced across sparsity weight

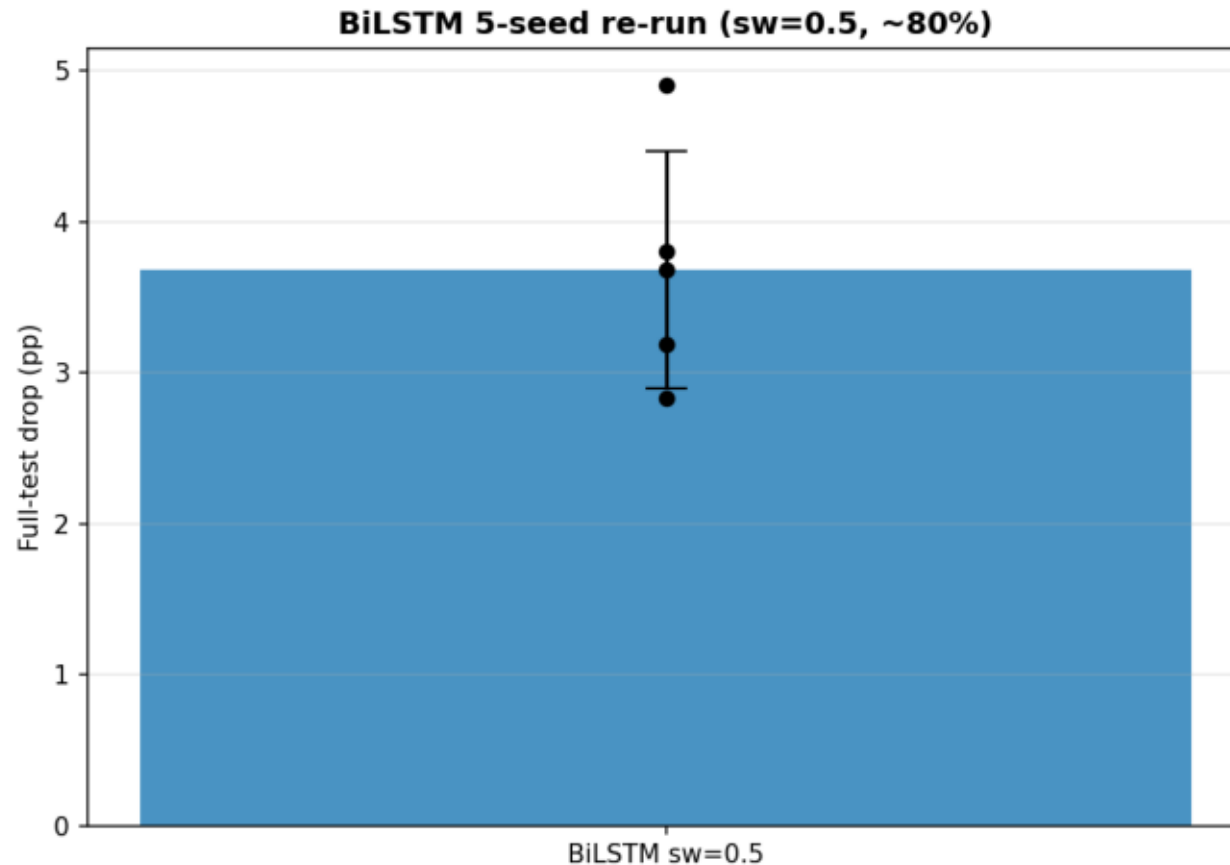


Pareto frontier (left) and accuracy drop vs % pruned (right). MNIST [1024,1024], baseline 98.06%.

- ▶ Activation pruning gives up first — sharp accuracy cliff past ~55% pruned.
- ▶ **BiLSTM stays within $\leq 1\text{pp}$ of baseline until ~72% pruned, and $\leq 4\text{pp}$ until ~80%.**
- ▶ Same trained model, same metric — the only thing that changes is the scoring function.
- ▶ Mechanism: BiLSTM sees the full weight vector per row; activation pruning summarises it to a single scalar.

The pruner is near-deterministic

Same checkpoint, fresh pruner seeds → essentially the same mask



5-seed re-run of the BiLSTM pruner at the ~80% operating point.

- ▶ 5 seeds, sw=0.5, ~80% pruned: drop = 3.68 ± 0.79 pp.
- ▶ Mask dead/alive ratio $\sigma = 0.007$ across seeds — essentially the same neurons get killed every time.
- ▶ Implies redundancy is a fixed property of the trained weights, not an optimization artefact.
- ▶ Lets us trust a single seed for downstream sweeps (verified, didn't assume).

Prunability scales with overparameterisation

Same MNIST task, sweep hidden dim 32 → 2048 (both layers)

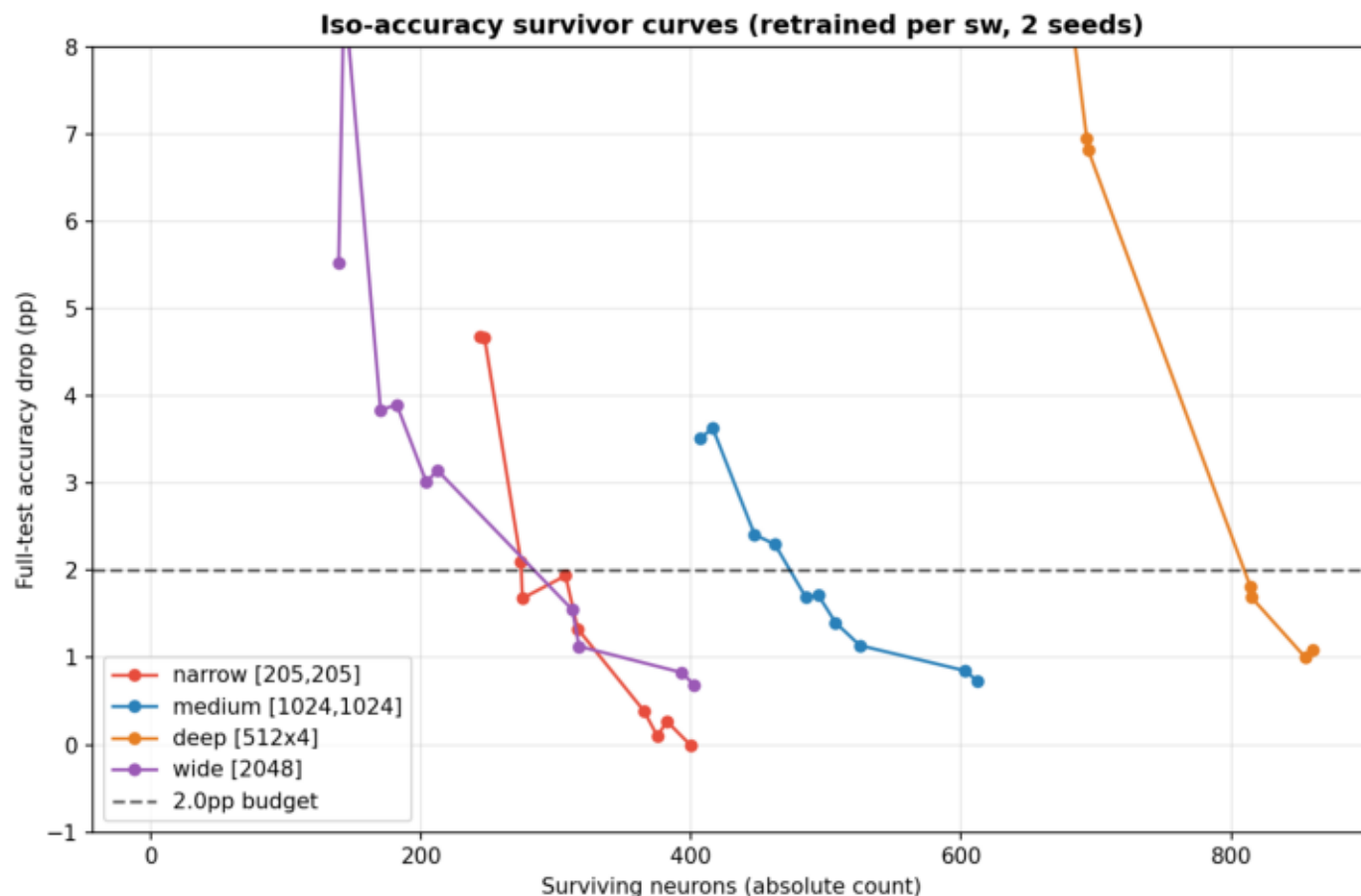


- ▶ **dim=32 → 8% pruned • dim=2048 → 51% pruned.**
- ▶ Test accuracy plateau is already reached by dim=128 — extra capacity buys redundancy, not accuracy.
- ▶ **Consistent with the lottery ticket picture: larger nets contain more 'spare' sub-networks the pruner can lock in on.**
- ▶ Sets up the deeper question — is it size, or is it WIDTH specifically? (Next slide.)

% nodes pruned (left) and pre-pruning test accuracy (right) vs hidden dim.

Width concentrates redundancy; depth is load-bearing

4 MNIST architectures, all ~2048 hidden neurons, iso-accuracy at $\leq 2\text{pp}$ drop



Each architecture's full accuracy-drop vs surviving-neurons curve; 2pp budget marked.

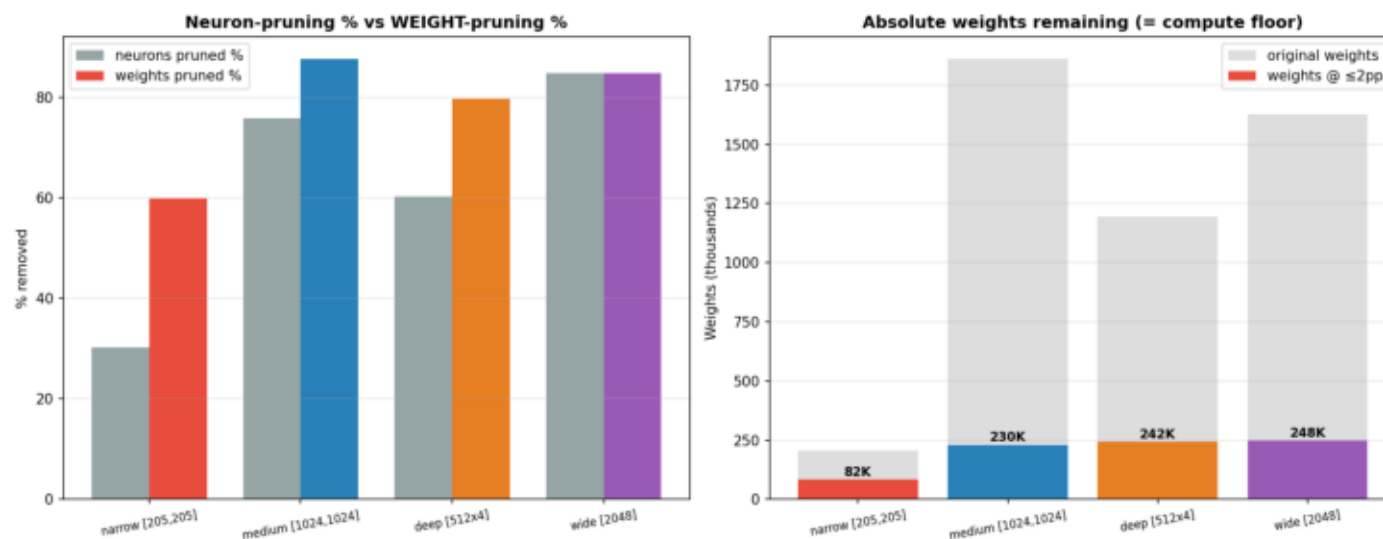
Survivors @ $\leq 2\text{pp}$ drop (pruner retrained per λ)

architecture	survivors	% pruned
wide [2048]	314	85 %
medium [1024,1024]	490	76 %
deep [512x4]	814	60 %
narrow [205,205]	296	28 %

- **Prunability monotonic in WIDTH.**
- Absolute survivors grow with DEPTH — a deeper net needs more live neurons to do its work.
- **Width = cheap parallel redundancy. Depth = sequential load-bearing capacity.**

Weights — not neurons — are the conserved quantity

The three 2048-neuron architectures all converge to the same compute floor

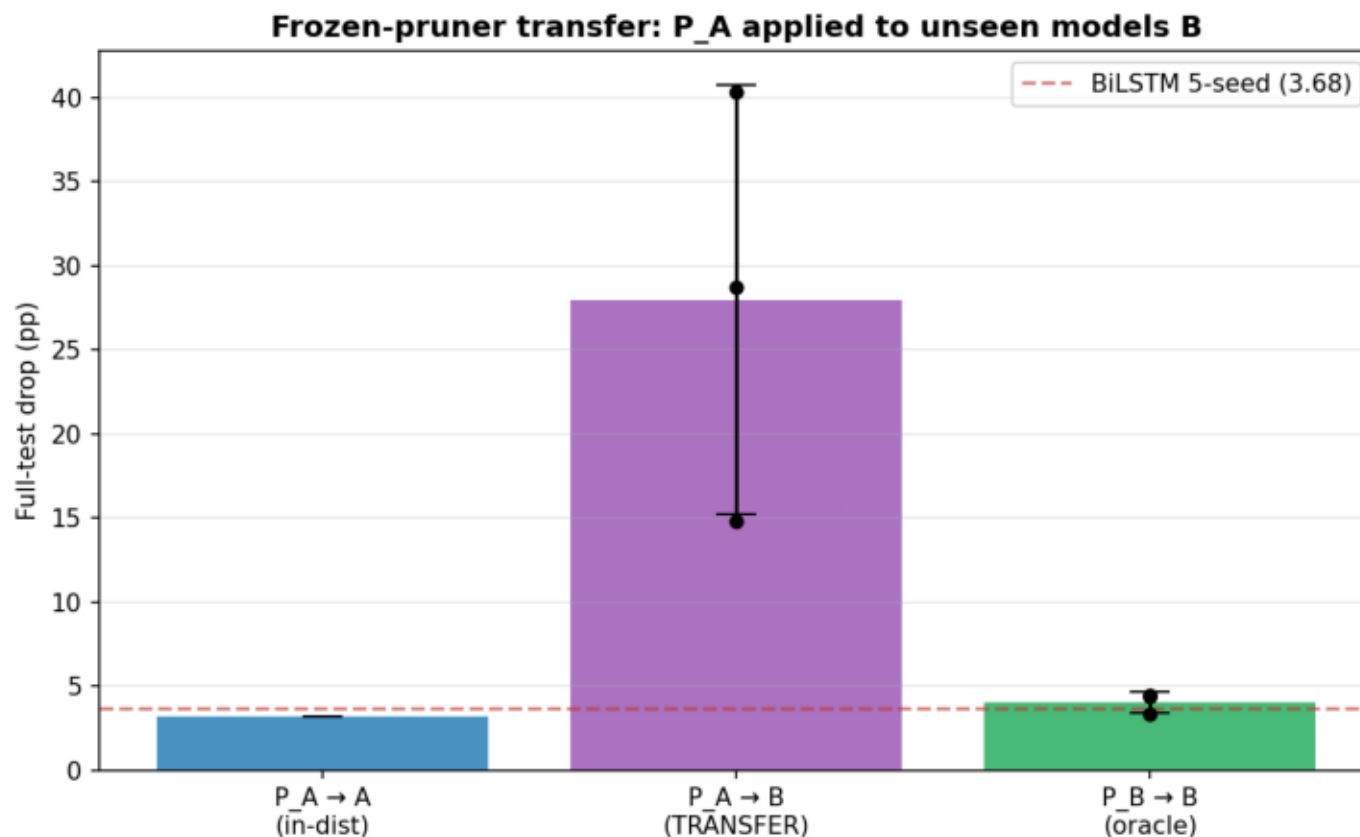


Left: neuron-% vs weight-% pruned. Right: absolute weights remaining (coloured) vs original size (grey).

- ▶ At $\leq 2pp$ drop, every 2048-neuron MNIST architecture ends up at ~240K weights ($\sigma = 8\%$):
- ▶ medium $\rightarrow$ 230K deep $\rightarrow$ 242K
wide $\rightarrow$ 248K
- ▶ Despite very different neuron survivors (490 / 814 / 314) and starting sizes (1.19M – 1.86M).
- ▶
- ▶ **Reading: ~240K MACs is an architecture-invariant compute floor for MNIST @ 2pp drop.**
- ▶ **But pruning-from-big $\neq$ training-small. A network trained at 410 neurons (narrow) reaches the SAME accuracy with only 82K weights — $\sim 3\times$ fewer. Pruning leaves structural redundancy on the table.**

The method transfers — the trained pruner does not

Freeze pruner P_A trained on model A ; apply directly to model B

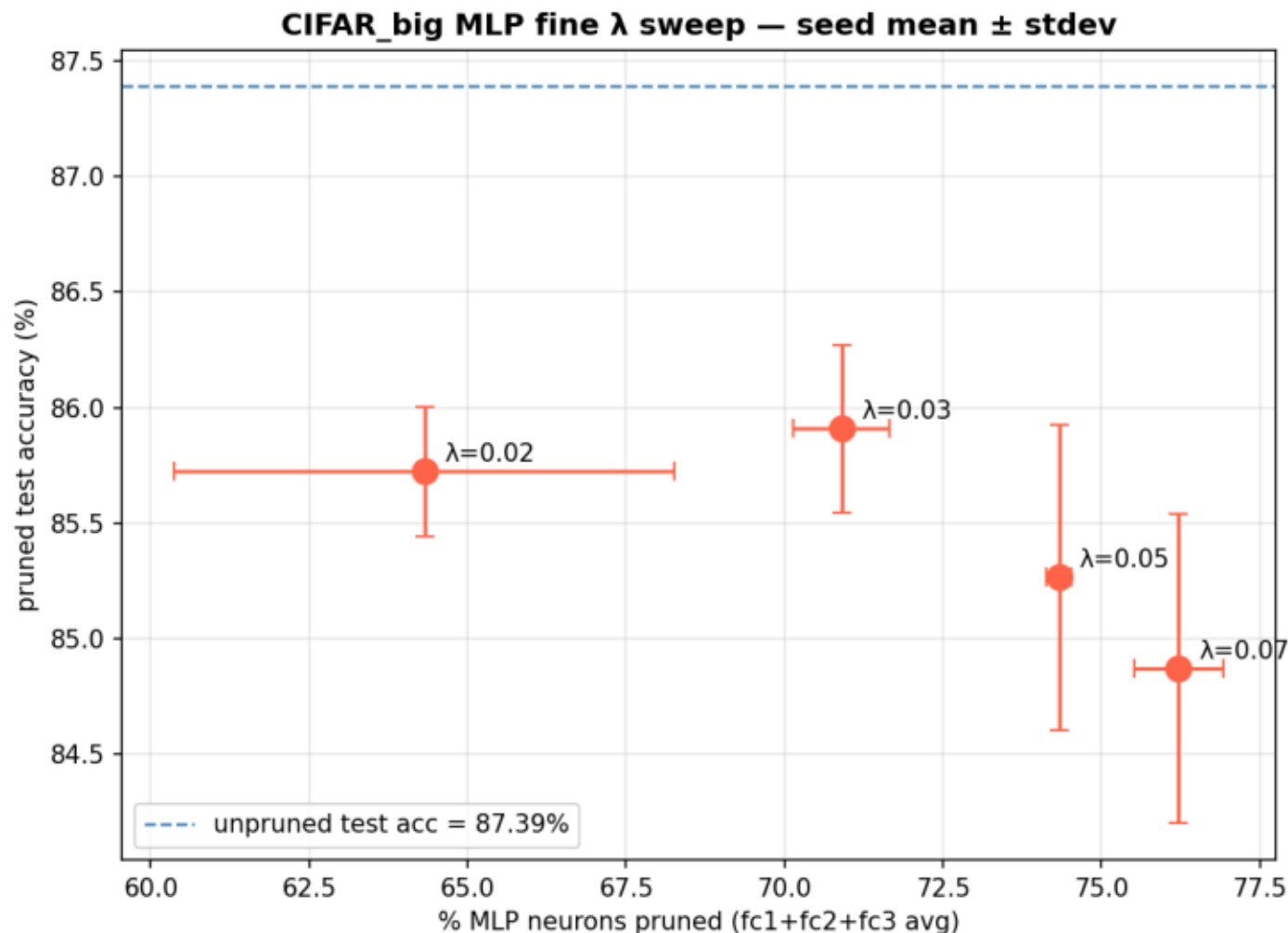


- ▶ $P_A \rightarrow A$ (in-distribution): ~3.3 pp drop
- ▶ $P_A \rightarrow B$ (frozen transfer): ~28 pp drop — catastrophic.
- ▶ $P_B \rightarrow B$ (oracle, retrain): ~4 pp drop
- ▶
- ▶ **Implication: redundancy is real and findable in every network, but its LOCATION is per-network — the permutation frame differs.**
- ▶ Re-running the pruner is cheap; expect to re-run it. What transfers is the procedure, not the answer.

Pruner trained on A then applied (a) back to A , (b) frozen to unseen B , (c) freshly trained on B .

Holds on a larger task — CIFAR-10 conv-net MLP head

CIFAR_big (10.4M params, baseline 87.39%); pruner targets the 3-layer FC head

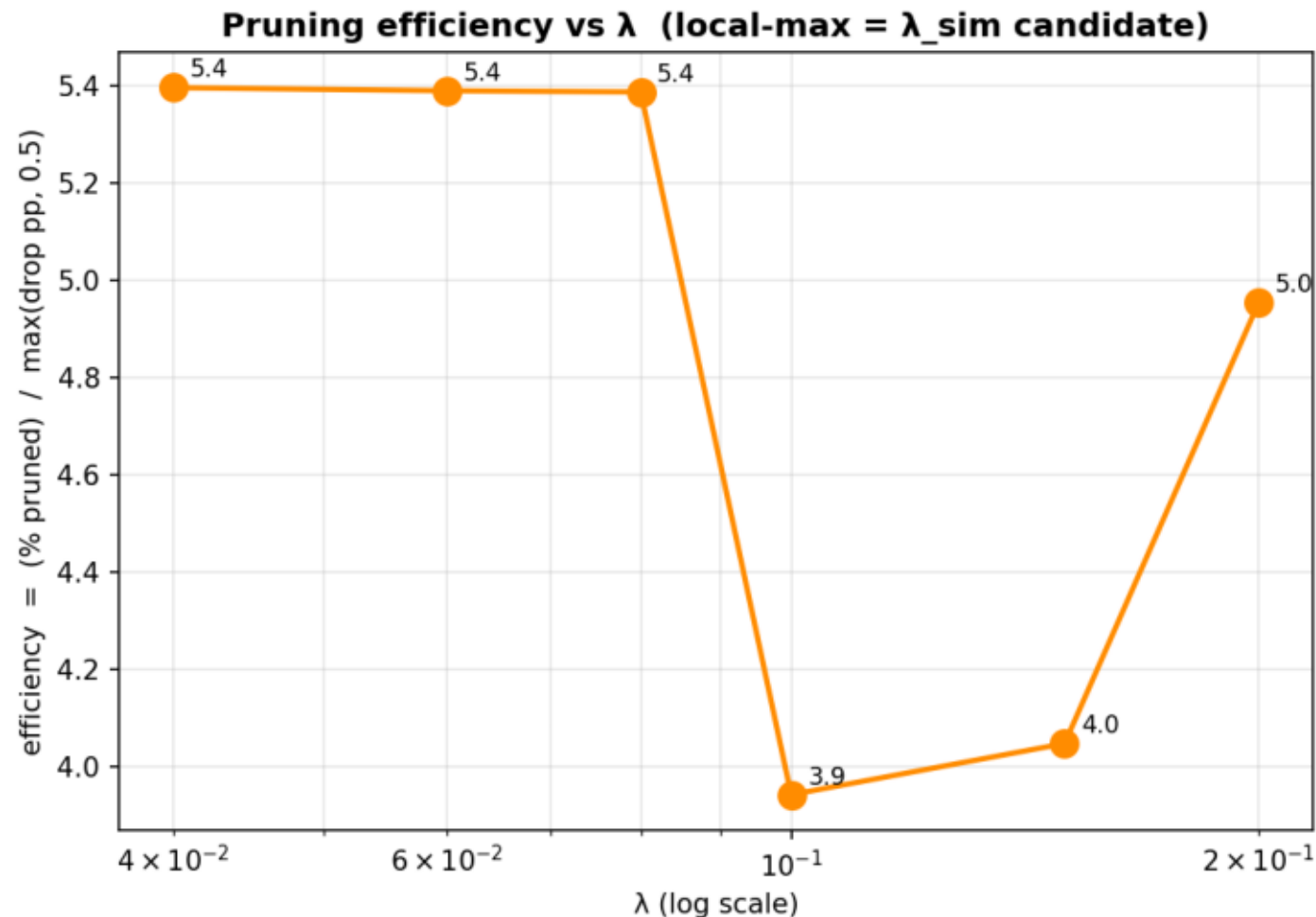


Pruned test accuracy vs % MLP-head neurons pruned, 4 λ values $\times$ 3 seeds.

- ▶ $\lambda = 0.03$ Pareto knee: 70.9 ± 0.8 % pruned at -1.48 ± 0.36 pp. 3 seeds, stable.
- ▶ Layer allocation mirrors the MNIST width finding:
 - ▶ fc1 (widest, 1024) $\rightarrow$ $\sim 17\%$ kept
 - ▶ fc2 (512) $\rightarrow$ $\sim 35\%$ kept
 - ▶ fc3 (256, last hidden) $\rightarrow$ $\sim 36\%$ kept — load-bearing floor.
- ▶ Widest layer prunes hardest in fraction terms; the layer right before output is incompressible. The width / depth axis shows up INSIDE one network too.

Two dynamical regimes — saturated narrow nets are bimodal in λ

CIFAR LeNet (63K, baseline 64.84%); pruning efficiency vs λ , 3 seeds $\times$ 15 ep

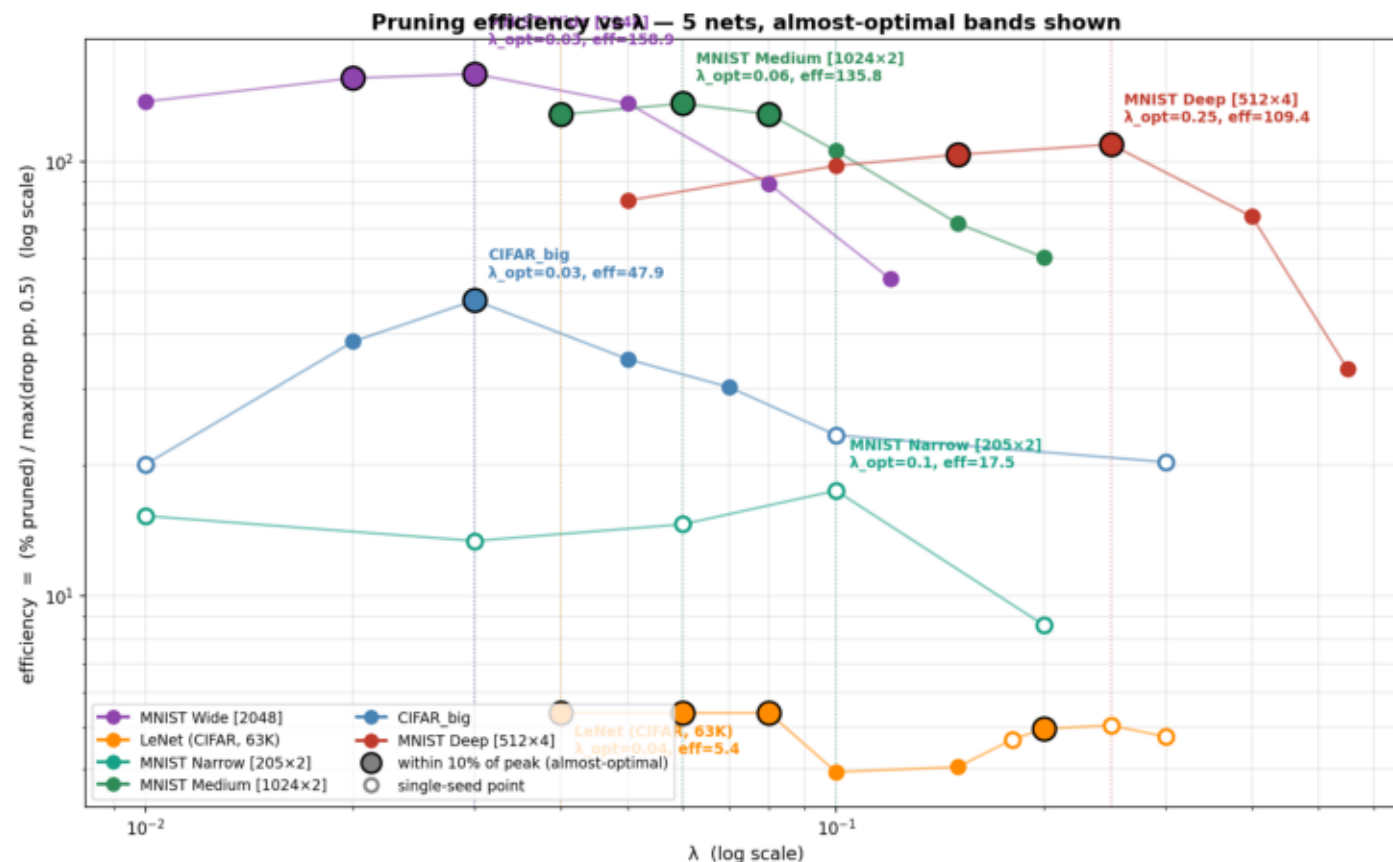


Efficiency = (% pruned) / max(drop pp, 0.5); local-maxima mark λ_{sim} candidates.

- ▶ Low- λ plateau ($\approx 0.04 - 0.08$): the SEQUENTIAL regime — gates close greedy layer-by-layer.
- ▶ **Sharp dip at $\lambda \approx 0.10$: half-committed dynamics, STRUCTURAL — persists at 15 epochs.**
- ▶ **Recovery at $\lambda \approx 0.25$: the SIMULTANEOUS regime — every layer commits in parallel.**
- ▶ **Mechanism: a gate closes when $\lambda/(N \cdot S_{\ell}) > -\partial \text{CE} / \partial g_i$. Low λ satisfies this only for the most-redundant layer; high λ satisfies it everywhere at once.**
- ▶ Standing instruction: LeNet λ sweeps must cover BOTH regions ($\approx 0.04 - 0.08$ and $\approx 0.20 - 0.30$).

Across architectures and tasks — λ_{opt} is NOT monotonic in size

3 base networks · 2 tasks · matched 15-ep × 3-seed protocol



Efficiency vs λ for 5 nets (incl. shape-study variants); almost-optimal bands shaded.

Peak efficiency per network

net	params	λ_{opt}	peak eff
CIFAR LeNet	63K	0.04	5.4
MNIST [1024 ²]	1.86M	0.06	135.8
CIFAR_big	10.4M	0.03	47.9

- ▶ λ_{opt} lives in [0.03, 0.06] despite 165× size shrink AND a task swap.
- ▶ Original ' λ_{opt} scales with size' hypothesis — refuted.
- ▶ Empirical regularity worth testing: λ_{opt} · N_layers $\approx 0.097 \pm 0.02$.

Takeaways

Weights alone encode redundancy.

A BiLSTM reading weight matrices prunes ~80% of MNIST MLPs at <4pp drop, near-deterministically across seeds.

Width concentrates redundancy; depth is load-bearing.

Same 2048-neuron budget: wide [2048] prunes 85%, medium 76%, deep 60%. All three compress to ~240K weights — the architecture-invariant compute floor on MNIST.

The method transfers, the weights don't.

A frozen pruner from one network applied to another: 28pp drop. Retrain the same architecture on B: ~4pp. Redundancy is per-network in the permutation frame.